

Arbres de régression et de classification

CART — Classification And Regression Trees

STA211 — Apprentissage supervisé — Fiche récapitulative

6 juin 2026

Table des matières

1	Présentation	2
2	Architecture d'un arbre	2
3	Algorithme de construction	3
3.1	Principe du partitionnement récursif	3
3.2	Critères de division	3
3.2.1	Régression (critère des moindres carrés)	3
3.2.2	Classification	3
4	Élagage (pruning)	3
5	Exemple numérique pas à pas	4
5.1	Données	4
5.2	Recherche de la première division	4
5.3	Division suivante	5
5.4	Arbre final	5
6	Code Python	5
7	Code R	7

1 Présentation

Les **arbres CART** (Breiman et al., 1984) sont des méthodes d'apprentissage **supervisé non-paramétriques**. Ils partitionnent l'espace des variables explicatives $\mathcal{X}$ en régions et associent une prédiction simple (moyenne ou mode) à chaque région.

Types d'arbres :

- **Arbre de régression** : Y quantitative $\rightarrow$ prédiction = moyenne des Y dans la région.
- **Arbre de classification** : Y qualitative $\rightarrow$ prédiction = modalité majoritaire dans la région.

Avantages :

- Interprétabilité (règles de décision lisibles)
- Gère naturellement les variables mixtes (quantitatives + qualitatives)
- Pas d'hypothèse sur la distribution des données

Inconvénients :

- Forte variance (instabilité) : un petit changement dans les données peut complètement changer l'arbre
- Biais de prédiction (souvent moins performant que forêts aléatoires / boosting)

2 Architecture d'un arbre

Un arbre est composé de :

- **Racine** : premier nœud contenant tous les individus.
- **Nœuds internes (fils)** : division selon une variable X_j et un seuil s .
- **Feuilles (nœuds terminaux)** : régions $R_1, \dots, R_M$ de l'espace des entrées.

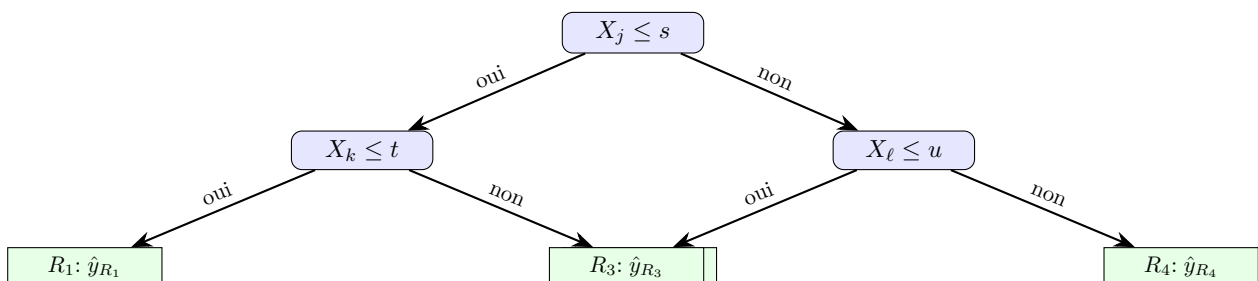
La valeur prédite dans une région R_m est :

$$\text{Régression : } \hat{y}_{R_m} = \frac{1}{n_m} \sum_{i \in R_m} y_i, \quad \text{Classification : } \hat{y}_{R_m} = \arg \max_k \hat{p}_{mk}$$

où $\hat{p}_{mk}$ est la proportion d'individus de classe k dans la région R_m .

Le modèle s'écrit :

$$f(\mathbf{x}) = \sum_{m=1}^M \hat{y}_{R_m} \cdot \mathbf{1}_{\{\mathbf{x} \in R_m\}}$$



3 Algorithme de construction

3.1 Principe du partitionnement récursif

On procède **de haut en bas**, par divisions binaires successives (*recursive binary splitting*) :

1. Partir de la racine (tous les individus).
2. Pour chaque nœud, chercher la variable X_j et le seuil s qui minimisent le **critère d'hétérogénéité**.
3. Diviser le nœud en deux fils selon $X_j < s$ et $X_j \geq s$.
4. Répéter sur chaque nœud fils jusqu'à un critère d'arrêt (homogénéité, taille minimale, profondeur max).

3.2 Critères de division

3.2.1 Régression (critère des moindres carrés)

On minimise la somme des carrés résiduels (RSS) dans les deux nœuds fils :

$$\min_{j,s} \left[\sum_{i:x_{ij} < s} (y_i - \bar{y}_g)^2 + \sum_{i:x_{ij} \geq s} (y_i - \bar{y}_d)^2 \right]$$

où $\bar{y}_g$ et $\bar{y}_d$ sont les moyennes dans les nœuds gauche et droit.

3.2.2 Classification

On maximise la **pureté** des nœuds fils. Critères courants :

- **Erreur de classification** : $1 - \max_k \hat{p}_{mk}$
- **Indice de Gini** : $G_m = \sum_{k \neq k'} \hat{p}_{mk} \hat{p}_{mk'} = 1 - \sum_k \hat{p}_{mk}^2$
- **Entropie croisée** : $D_m = - \sum_k \hat{p}_{mk} \log \hat{p}_{mk}$

Le **Gini** est le plus utilisé (CART). La division optimale minimise la moyenne pondérée du critère :

$$\min_{j,s} [n_g \cdot G_g + n_d \cdot G_d]$$

4 Élagage (pruning)

Un arbre complet (*full-grown tree*) surajuste presque toujours. On utilise l'**élagage par coût-complexité** (*cost-complexity pruning*) :

$$\text{Critère}_\alpha(A) = \underbrace{\sum_{m=1}^{|A|} \sum_{i \in R_m} (y_i - \hat{y}_{R_m})^2}_{\text{Erreur d'apprentissage}} + \alpha \cdot |A|$$

où $|A|$ est le nombre de feuilles de l'arbre A et $\alpha \geq 0$ un paramètre de régularisation.

- $\alpha = 0$: arbre complet (max de feuilles).
- α grand : arbre simple (peu de feuilles).
- On choisit α par **validation croisée** (souvent 10-fold CV).

5 Exemple numérique pas à pas

5.1 Données

Considérons $n = 6$ individus avec une variable explicative X et une réponse Y (régression) :

X	Y
1	2
2	3
3	8
4	9
5	4
6	5

5.2 Recherche de la première division

On teste chaque seuil s possible (entre 1 et 6).

Seuil $s = 1$ ($\mathbf{X} < 1$ vs $\mathbf{X} \geq 1$) :

- Nœud gauche : $\emptyset$ (impossible — nœud vide)
- Nœud droit : tous les individus

Ce seuil n'est pas admissible (nœud vide).

Seuil $s = 2$ ($\mathbf{X} < 2$ vs $\mathbf{X} \geq 2$) :

- Gauche : $\{1\}$, $\bar{y}_g = 2$, $\text{RSS}_g = 0$
- Droite : $\{2, 3, 4, 5, 6\}$, $\bar{y}_d = \frac{3+8+9+4+5}{5} = 5,8$, $\text{RSS}_d = (3 - 5,8)^2 + (8 - 5,8)^2 + (9 - 5,8)^2 + (4 - 5,8)^2 + (5 - 5,8)^2$
- $\text{RSS}_d = 7,84 + 4,84 + 10,24 + 3,24 + 0,64 = 26,8$
- $\text{RSS total} = 0 + 26,8 = 26,8$

Seuil $s = 3$ ($\mathbf{X} < 3$ vs $\mathbf{X} \geq 3$) :

- Gauche : $\{1, 2\}$, $\bar{y}_g = 2,5$, $\text{RSS}_g = (2 - 2,5)^2 + (3 - 2,5)^2 = 0,5$
- Droite : $\{3, 4, 5, 6\}$, $\bar{y}_d = \frac{8+9+4+5}{4} = 6,5$, $\text{RSS}_d = (8 - 6,5)^2 + (9 - 6,5)^2 + (4 - 6,5)^2 + (5 - 6,5)^2$
- $\text{RSS}_d = 2,25 + 6,25 + 6,25 + 2,25 = 17$
- $\text{RSS total} = 0,5 + 17 = 17,5$

Seuil $s = 4$ ($\mathbf{X} < 4$ vs $\mathbf{X} \geq 4$) :

- Gauche : $\{1, 2, 3\}$, $\bar{y}_g = \frac{2+3+8}{3} \approx 4,33$, $\text{RSS}_g \approx 20,67$
- Droite : $\{4, 5, 6\}$, $\bar{y}_d = \frac{9+4+5}{3} = 6$, $\text{RSS}_d = 14$
- $\text{RSS total} \approx 34,67$

Seuil $s = 5$ ($\mathbf{X} < 5$ vs $\mathbf{X} \geq 5$) :

- Gauche : $\{1, 2, 3, 4\}$, $\bar{y}_g = 5,5$, $\text{RSS}_g = 29$
- Droite : $\{5, 6\}$, $\bar{y}_d = 4,5$, $\text{RSS}_d = 0,5$
- $\text{RSS total} = 29,5$

Bilan : Le seuil optimal est $s = 3$ ($\text{RSS minimal} = 17,5$). La première division est donc $X < 3$ vs $X \geq 3$.

5.3 Division suivante

On répète le processus sur chaque nœud fils.

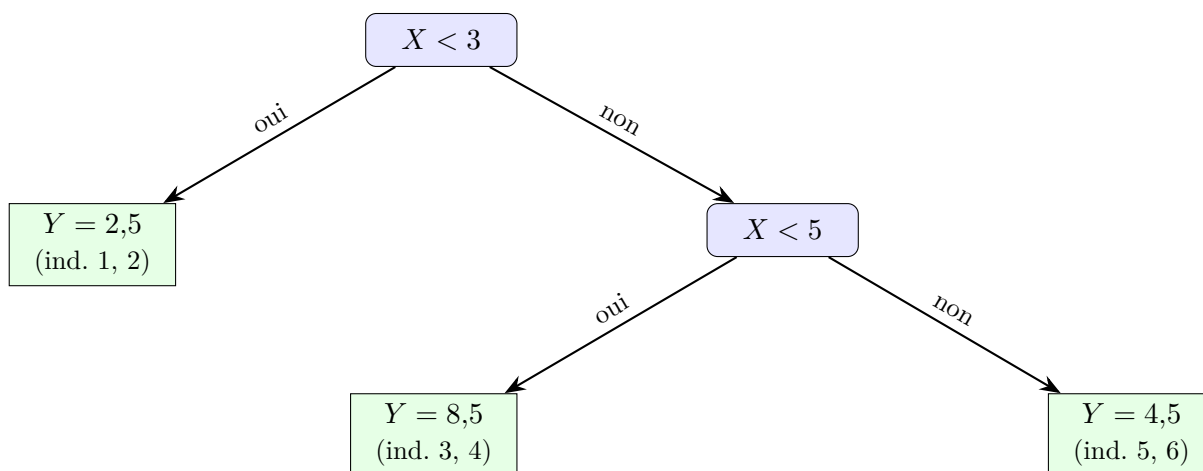
Nœud gauche (individus 1, 2) : $\bar{y} = 2,5$, variance intra = 0,25. Pas de gain significatif à diviser (trop petit). On s'arrête.

Nœud droit (individus 3, 4, 5, 6) : On teste $s = 5$ (seuil optimal après calculs).

- Gauche : $\{3, 4\}$, $\bar{y} = 8,5$, RSS = 0,5
- Droite : $\{5, 6\}$, $\bar{y} = 4,5$, RSS = 0,5
- RSS total = 1

Division retenue : $X < 5$ vs $X \geq 5$.

5.4 Arbre final



Prédictions :

$$\hat{y} = \begin{cases} 2,5 & \text{si } X < 3 \\ 8,5 & \text{si } 3 \leq X < 5 \\ 4,5 & \text{si } X \geq 5 \end{cases}$$

6 Code Python

```

1 from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier
2 from sklearn.tree import plot_tree
3 from sklearn.model_selection import cross_val_score
4 import matplotlib.pyplot as plt
5 import pandas as pd
6 import numpy as np
7
8 # --- Arbre de regression ---
9 np.random.seed(42)
10 X = np.linspace(0, 10, 100).reshape(-1, 1)
11 y = np.sin(X).ravel() + np.random.randn(100) * 0.1
12
13 tree_reg = DecisionTreeRegressor(max_depth=3, min_samples_leaf=5)
14 tree_reg.fit(X, y)
15
16 # Visualisation
17 plt.figure(figsize=(12, 6))
18 plot_tree(tree_reg, filled=True, feature_names=['X'])
19 plt.title("Arbre de regression")
20 plt.show()

```

```

21
22 # Prediction
23 X_new = np.array([[5.5]])
24 print(f"Prediction : {tree_reg.predict(X_new)[0]:.3f}")

```

Listing 1 – Arbre de régression/classification avec scikit-learn

```

1 from sklearn.datasets import load_iris
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import confusion_matrix, classification_report
5
6 iris = load_iris()
7 X, y = iris.data, iris.target
8
9 # Separation train/test
10 X_train, X_test, y_train, y_test = train_test_split(
11     X, y, test_size=0.3, random_state=42)
12
13 # Arbre (Gini par défaut, profondeur max 4)
14 tree_clf = DecisionTreeClassifier(
15     max_depth=4, criterion='gini', min_samples_leaf=5)
16 tree_clf.fit(X_train, y_train)
17
18 # Predictions
19 y_pred = tree_clf.predict(X_test)
20 print(confusion_matrix(y_test, y_pred))
21 print(classification_report(y_test, y_pred,
22     target_names=iris.target_names))
23
24 # Importance des variables
25 for name, imp in zip(iris.feature_names, tree_clf.feature_importances_):
26     print(f"{name}: {imp:.3f}")

```

Listing 2 – Arbre de classification — Iris

```

1 # Chemin d'élague (cost complexity pruning path)
2 path = tree_clf.cost_complexity_pruning_path(X_train, y_train)
3 ccp_alphas, impurities = path.ccp_alphas, path.impurities
4
5 # Selection par validation croisee
6 from sklearn.model_selection import GridSearchCV
7
8 param_grid = {'ccp_alpha': ccp_alphas}
9 grid = GridSearchCV(
10     DecisionTreeClassifier(random_state=42),
11     param_grid, cv=5
12 )
13 grid.fit(X_train, y_train)
14
15 print(f"Meilleur alpha : {grid.best_params_['ccp_alpha']:.4f}")
16 print(f"Meilleur score : {grid.best_score_:.3f}")
17
18 # Arbre elague optimal
19 best_tree = grid.best_estimator_
20 print(f"Nombre de feuilles : {best_tree.get_n_leaves()}")

```

Listing 3 – Élagage par coût-complexité

7 Code R

```

1 # install.packages("rpart")
2 # install.packages("rpart.plot")
3 library(rpart)
4 library(rpart.plot)
5
6 # Arbre de classification sur iris
7 data(iris)
8 set.seed(42)
9
10 # Separation train/test
11 idx <- sample(1:nrow(iris), 0.7 * nrow(iris))
12 train <- iris[idx, ]
13 test <- iris[-idx, ]
14
15 # Modele
16 arbre <- rpart(Species ~ ., data = train,
17               method = "class",
18               control = rpart.control(minsplit = 5,
19                                     maxdepth = 4,
20                                     cp = 0.01))
21
22 # Visualisation
23 rpart.plot(arbre, type = 2, extra = 104,
24           main = "Arbre de classification (Iris)")
25
26 # Predictions
27 pred <- predict(arbre, test, type = "class")
28 table(pred, test$Species)
29
30 # Elagage par cp (complexity parameter)
31 print(arbre$cptable) # table cout-complexite
32 best_cp <- arbre$cptable[which.min(arbre$cptable[, "xerror"]), "CP"]
33 arbre_elague <- prune(arbre, cp = best_cp)
34 rpart.plot(arbre_elague, type = 2, extra = 104,
35           main = "Arbre elague optimal")

```

Listing 4 – Arbre de classification avec rpart

```

1 # Arbre de regression
2 X <- seq(0, 10, length.out = 100)
3 y <- sin(X) + rnorm(100, sd = 0.1)
4 data_reg <- data.frame(X = X, Y = y)
5
6 tree_reg <- rpart(Y ~ X, data = data_reg,
7                 method = "anova",
8                 control = rpart.control(minsplit = 5,
9                                       cp = 0.001))
10 rpart.plot(tree_reg, type = 2, main = "Arbre de regression")
11
12 # Prediction
13 predict(tree_reg, newdata = data.frame(X = 5.5))

```

Listing 5 – Arbre de régression

QCM — Vérifiez vos connaissances

Pour chaque question, choisissez la bonne réponse. Les réponses sont en page 10.

1. CART est l'acronyme de :
 - (a) Classification And Regression Trees
 - (b) Categorical Automatic Rule Testing
 - (c) Clustering And Recursive Trees
 - (d) Continuous Asymptotic Regression Trees
2. La construction d'un arbre CART procède par :
 - (a) divisions aléatoires de l'espace
 - (b) partitionnement récursif binaire descendant
 - (c) fusion ascendante de régions
 - (d) optimisation globale par programmation dynamique
3. En régression, le critère de division est :
 - (a) l'indice de Gini
 - (b) l'entropie croisée
 - (c) la somme des carrés résiduels (RSS)
 - (d) le R^2
4. En classification, le critère de division le plus utilisé dans CART est :
 - (a) l'erreur de classification
 - (b) l'indice de Gini
 - (c) l'entropie croisée
 - (d) le critère d'Akaike (AIC)
5. L'élitage par coût-complexité vise à :
 - (a) augmenter le nombre de feuilles
 - (b) réduire la variance en pénalisant la complexité
 - (c) accélérer l'apprentissage
 - (d) équilibrer les classes
6. Dans le critère coût-complexité $\text{Err}(A) + \alpha|A|$, $|A|$ représente :
 - (a) le nombre de nœuds internes
 - (b) le nombre de feuilles
 - (c) la profondeur de l'arbre
 - (d) le nombre de variables
7. Pour choisir le paramètre d'élitage α , on utilise :
 - (a) un test de Student
 - (b) la validation croisée
 - (c) le critère AIC
 - (d) le test du chi-deux
8. Les arbres CART sont dits **non-paramétriques** car :
 - (a) ils n'ont pas de paramètres
 - (b) ils ne font pas d'hypothèse sur la distribution des données
 - (c) ils utilisent des paramètres non linéaires
 - (d) ils ne nécessitent pas d'apprentissage
9. Un inconvénient majeur des arbres CART est :
 - (a) leur forte variance (instabilité)
 - (b) leur incapacité à gérer les variables qualitatives

- (c) leur temps d'apprentissage très long
 - (d) leur manque d'interprétabilité
10. Quel package R implémente les arbres CART ?
- (a) randomForest
 - (b) rpart
 - (c) ggplot2
 - (d) caret

Réponses au QCM

N°	Réponse	Explication
1	a	CART = Classification And Regression Trees (Breiman et al., 1984).
2	b	L'algorithme divise récursivement chaque nœud en deux, de la racine vers les feuilles.
3	c	En régression, on minimise la somme des carrés résiduels (RSS) dans les nœuds fils.
4	b	L'indice de Gini $1 - \sum \hat{p}_{mk}^2$ est le critère par défaut de CART pour la classification.
5	b	L'élagage réduit la variance en pénalisant un grand nombre de feuilles (contrôle du surajustement).
6	b	$ A $ est le nombre de feuilles (nœuds terminaux) de l'arbre.
7	b	On choisit α par validation croisée (10-fold CV typiquement).
8	b	Aucune hypothèse sur la distribution des données n'est nécessaire (contrairement à la régression linéaire).
9	a	Un petit changement dans les données peut complètement modifier l'arbre (forte variance).
10	b	Le package <code>rpart</code> implémente les arbres CART en R.

Fin de la fiche.